

TỔNG QUAN VỀ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG (OOP) TRONG JAVA

1. Giới thiệu chung

Lập trình hướng đối tượng (Object-Oriented Programming - OOP) là một mô hình lập trình dựa trên khái niệm "đối tượng" (object), thay vì chỉ dựa trên các hàm và logic như lập trình thủ tục (procedural programming).

Trong mô hình này, một phần mềm được thiết kế xoay quanh dữ liệu hoặc các đối tượng hơn là các chức năng và logic. Một đối tượng bao gồm hai thành phần chính:

- Dữ liệu (Data):** Được gọi là thuộc tính (attributes) hoặc trường (fields).
- Mã nguồn (Code):** Được gọi là phương thức (methods) hoặc hành vi (behaviors).

Java là một trong những ngôn ngữ lập trình phổ biến nhất thể hiện rõ nét tư duy hướng đối tượng. Hầu hết mọi thứ trong Java đều gắn liền với Class (Lớp) và Object (Đối tượng).

2. Các khái niệm cốt lõi: Lớp và Đối tượng

Để hiểu về OOP, trước tiên cần phân biệt rõ hai khái niệm nền tảng sau:

2.1. Lớp (Class)

Lớp là một khuôn mẫu (template) hoặc bản thiết kế (blueprint) dùng để tạo ra các đối tượng. Lớp định nghĩa các thuộc tính và hành vi chung mà mọi đối tượng thuộc lớp đó đều sở hữu.

Ví dụ: Nếu ta có một lớp `SinhVien`, lớp này sẽ định nghĩa rằng mọi sinh viên đều cần có `mã số`, `tên`, và hành vi `học bài`.

2.2. Đối tượng (Object)

Đối tượng là một thực thể cụ thể (instance) được tạo ra từ một lớp. Mỗi đối tượng có trạng thái riêng (giá trị của các thuộc tính) và hành vi riêng dựa trên định nghĩa của lớp.

Ví dụ: Từ lớp `SinhVien`, ta tạo ra đối tượng `sv1` có tên là "Nguyễn Văn A" và đối tượng `sv2` có tên là "Trần Thị B".

Cú pháp khai báo trong Java:

```
// Định nghĩa Lớp
public class Car {
    String color; // Thuộc tính

    void run() { // Phương thức
        System.out.println("Xe đang chạy");
    }
}

// Tạo Đối tượng
public class Main {
    public static void main(String[] args) {
        Car myCar = new Car(); // Khởi tạo đối tượng myCar từ lớp Car
        myCar.color = "Red";
        myCar.run();
    }
}
```

3. Bốn nguyên lý cơ bản của OOP

Sức mạnh của lập trình hướng đối tượng nằm ở 4 nguyên lý trụ cột sau:

3.1. Tính đóng gói (Encapsulation)

Tính đóng gói là kỹ thuật che giấu thông tin chi tiết bên trong của đối tượng và chỉ cung cấp những gì cần thiết ra bên ngoài. Điều này giúp bảo vệ dữ liệu không bị thay đổi tùy tiện từ bên ngoài.

Trong Java, tính đóng gói thường được thực hiện thông qua các phạm vi truy cập (Access Modifiers) như `private`, `protected`, `public`. Dữ liệu thường được để `private` và truy cập thông qua các phương thức `getter` và `setter` công khai.

Ví dụ:

```
public class Person {
    private String name; // Dữ liệu bị ẩn

    // Phương thức công khai để truy cập dữ liệu
    public String getName() {
        return name;
    }

    public void setName(String newName) {
        this.name = newName;
    }
}
```

3.2. Tính kế thừa (Inheritance)

Tính kế thừa cho phép một lớp mới (lớp con - subclass) kế thừa các thuộc tính và phương thức của một lớp đã tồn tại (lớp cha - superclass). Điều này giúp tái sử dụng mã nguồn và tạo nên cấu trúc phân cấp rõ ràng.

Trong Java, từ khóa **extends** được sử dụng để thực hiện kế thừa.

Ví dụ:

```
class Animal {
    void eat() {
        System.out.println("Đang ăn...");
    }
}

// Dog kế thừa từ Animal
class Dog extends Animal {
    void bark() {
        System.out.println("Gâu gâu");
    }
}
```

3.3. Tính đa hình (Polymorphism)

Đa hình có nghĩa là "nhiều hình thái". Tính chất này cho phép một hành động có thể được thực hiện theo nhiều cách khác nhau tùy thuộc vào đối tượng thực hiện nó.

Trong Java, đa hình thể hiện qua hai hình thức chính:

- **Nạp chồng (Overloading):** Cùng tên phương thức nhưng khác tham số (xảy ra trong cùng một lớp).
- **Ghi đè (Overriding):** Lớp con định nghĩa lại phương thức đã có ở lớp cha (xảy ra giữa hai lớp có quan hệ kế thừa).

Ví dụ về Ghi đè:

```
class Animal {
    public void sound() {
        System.out.println("Tiếng kêu động vật");
    }
}

class Cat extends Animal {
    @Override
    public void sound() { // Ghi đè phương thức sound
        System.out.println("Meo meo");
    }
}
```

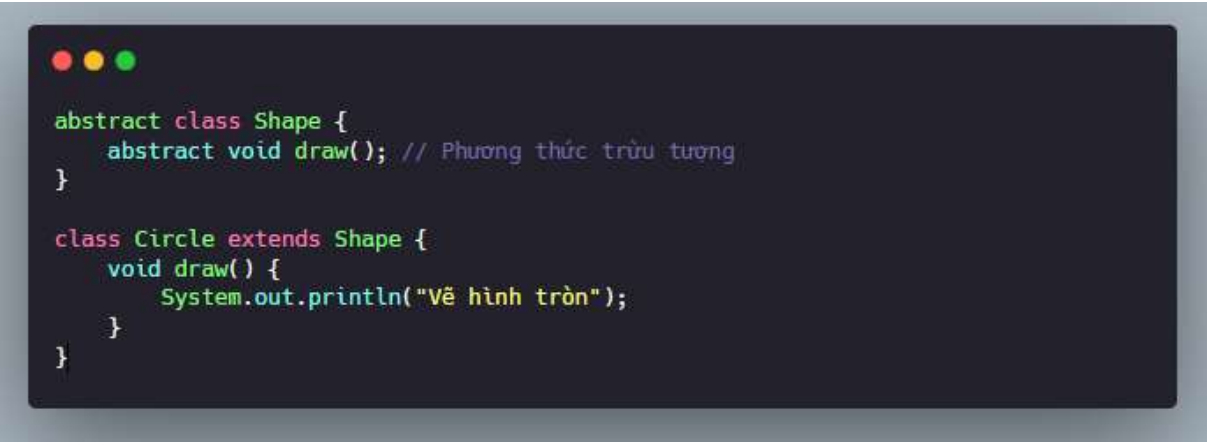
3.4. Tính trừu tượng (Abstraction)

Tính trừu tượng là tiến trình ẩn đi các chi tiết thực thi phức tạp và chỉ hiển thị các tính năng quan trọng của đối tượng cho người dùng. Nó giúp lập trình viên tập trung vào việc đối tượng "làm gì" thay vì "làm như thế nào".

Trong Java, tính trừu tượng đạt được thông qua:

- **Lớp trừu tượng (Abstract Class):** Lớp không thể khởi tạo đối tượng, có thể chứa phương thức trừu tượng (không có thân hàm).
- **Interface:** Một tập hợp các phương thức trừu tượng hoàn toàn (trước Java 8) quy định các hành vi mà lớp thực thi phải tuân thủ.

Ví dụ:



```
abstract class Shape {
    abstract void draw(); // Phương thức trừu tượng
}

class Circle extends Shape {
    void draw() {
        System.out.println("Vẽ hình tròn");
    }
}
```

4. Tại sao nên sử dụng OOP trong Java?

Việc áp dụng OOP mang lại nhiều lợi ích thiết thực trong phát triển phần mềm:

1. **Tái sử dụng mã nguồn:** Nhờ tính kế thừa, lập trình viên không cần viết lại mã cho các chức năng tương tự nhau.
2. **Dễ bảo trì và sửa lỗi:** Do tính đóng gói và module hóa, việc sửa lỗi ở một đối tượng ít gây ảnh hưởng dây chuyền đến các phần khác của hệ thống.
3. **Mô phỏng thế giới thực:** OOP giúp ánh xạ các nghiệp vụ thực tế vào phần mềm một cách tự nhiên và logic.
4. **Bảo mật dữ liệu:** Thông qua việc kiểm soát quyền truy cập dữ liệu (Encapsulation).

5. Kết luận

Lập trình hướng đối tượng là nền tảng không thể thiếu khi làm việc với Java. Việc nắm vững 4 nguyên lý: Đóng gói, Kế thừa, Đa hình và Trừu tượng không chỉ giúp lập trình viên viết mã hiệu quả hơn mà còn là cơ sở để thiết kế các hệ thống phần mềm lớn, phức tạp và bền vững.